

AUDIT REPORT

Bank Management System

Terminal-Based ATM Application in C

This report presents a comprehensive technical overview of the Bank Management System project, examining its architecture, module design, security model, and engineering decisions that make it a robust, production-quality terminal application.

Prepared for: Development Team Review
Version 1.0 | February 2026

1. Executive Summary

The Bank Management System is a fully-featured, terminal-based banking application written in C. It provides user registration and authentication, multi-account management, real-time transactions, account ownership transfers, and asynchronous inter-process notifications — all rendered through a custom ncurses TUI layer. The codebase is organised into clearly separated modules, persists all data in an embedded SQLite database, and implements modern password security using salted SHA-256 hashing via OpenSSL.

This audit covers the project's file architecture, the responsibility and design of each module, security and robustness measures, the IPC notification subsystem, the validation framework, and the TUI rendering engine.

Metric	Value	Notes
Source files (.c)	19	Across 4 sub-directories
Header files (.h)	11	One per module
External libraries	5	sqlite3, ncurses, menu, openssl, pthread
Database tables	2	users, accounts
Authentication model	Salted SHA-256	16-byte random salt per user
IPC mechanism	POSIX Named FIFO	Per-user pipe at /tmp/atm_fifo_<name>
Threading model	2 threads	Main UI thread + background notification listener

2. System Architecture

2.1 Layered Design

The application is structured into four horizontal layers, each with a clear and exclusive responsibility:

Layer	Module(s)	Responsibility
Presentation	tui.c, main_menu.c, init_menu.c, ui_helper.c	All ncurses rendering, modal dialogs, input fields, layout management.
Business Logic	system.c	All banking operations — create, update, transact, transfer, remove accounts.
Authentication	auth.c, init_menu.c	Login and registration with salted password hashing.

Persistence	database.c, database_oper.c	SQLite schema initialisation and connection helpers.
Utilities	utils/*	Hashing, IPC, string operations, terminal control, file I/O.
Validation	validators.c, get_valid_*.c	Pure, I/O-free input validation returning structured error strings.

2.2 Data Flow

The lifecycle of a user session follows a strict linear flow:

```
main()
├── initialize_database()    Creates tables if absent
├── tui_init()              Starts ncurses
├── initMenu()              Login / Register screen
│   └── getUserById()       Hydrates struct User from DB
├── pthread_create()        Spawns notification listener thread
├── mainMenu()              Event loop
│   ├── createNewAcc(u)
│   ├── updateAccount(u)
│   ├── checkAccountDetails(u)
│   ├── checkAllAccounts(u)
│   ├── makeTransaction(u)
│   ├── removeAccount(u)
│   └── transferOwnership(u) Also calls notifyUser()
```

3. Module Descriptions

3.1 main.c — Entry Point

Initialises all subsystems in the correct order: database first (so the schema exists before any query), then the TUI (so ncurses is live before any screen drawing), then the init menu (login/register), and finally the notification thread (so the username is known before the FIFO is created). The main thread ends by joining the notification thread on exit.

Design note Subsystem initialisation order is critical. Reversing any step — for example starting the TUI before the database — would cause undefined behaviour if the database fails to open and tries to print to stderr while ncurses is active.

3.2 auth.c — Authentication (Legacy)

Contains the original terminal-mode login and register implementation using raw termios for password masking and printf for output. This module is superseded by init_menu.c but remains in the codebase as the canonical implementation of getPassword() and isUsernameTaken(), which are still called by the newer ncurses-based login screen via forward extern declarations.

3.3 init_menu.c — Authentication UI (Active)

The active login and registration implementation. Uses the TUI layer for all input and output. The module implements two private screen functions — `_login_screen()` and `_register_screen()` — and exposes only `initMenu()` to the caller. All credential handling (hashing, salt generation, DB insert) is performed inline after the ncurses form completes, keeping the auth logic co-located with the UI that collects it. Also contains `getUserById()` which populates the struct `User` after successful login.

- Input validation: letters-only username, 8-12 character alphanumeric password
- Passwords confirmed by re-entry before hashing
- Duplicate username check via `isUsernameTaken()` before any DB write
- Successful login shows a modal, failed login shows a specific error modal

3.4 system.c — Banking Operations

The largest and most complex module. Implements all six banking operations as public functions. Each function opens its own SQLite connection, performs its operation, closes the connection, and delegates post-operation flow to `success()`. This design avoids keeping long-lived database connections open.

The module contains an extensive set of private form-field helpers (`_field_acct_nbr`, `_field_date`, `_field_country`, `_field_phone`, `_field_amount`) that wrap the validator functions with a retry loop and inline ncurses rendering. This means every form field validates input immediately in-place — the user sees a red error row appear inside the window without leaving the form.

Function	Key behaviour
<code>createNewAcc()</code>	Loads existing IDs to prevent duplicates; collects 5 fields inline; inserts via parameterised statement.
<code>updateAccount()</code>	Presents a two-field picker (country or phone); collects only the changed value.
<code>checkAccountDetails()</code>	Computes and displays interest projections per account type.
<code>checkAllAccounts()</code>	Renders a fixed-column table with alternating row colours, respecting terminal width.
<code>makeTransaction()</code>	Blocks fixed-term accounts. Prevents overdraft on withdrawal. Confirms new balance in modal.
<code>removeAccount()</code>	Enforces zero-balance requirement. Requires <code>tui_modal_confirm()</code> before deletion.
<code>transferOwnership()</code>	Verifies source ownership; validates target user exists; prevents ID collision; notifies recipient via FIFO.

3.5 ui/tui.c — Terminal User Interface Engine

The TUI module is the most technically sophisticated component of the project. It provides a complete abstraction over ncurses, isolating every other module from direct ncurses calls. Key subsystems within tui.c include:

UTF-8 Display Width (tui_dispw)

ncurses operates in bytes, not characters. When multi-byte UTF-8 sequences or wide emoji codepoints are present, `strlen()` returns an incorrect value for centering calculations. `tui_dispw()` decodes each UTF-8 sequence manually and assigns a display width of 2 for codepoints $\geq$ U+1F000 (emoji) and 1 for all others, producing correct centering on any terminal.

Async Notification System (tui_getch / tui_set_notification)

This subsystem solves a fundamental problem: how to interrupt the blocking main-thread input loop when a background thread receives a notification, without making any unsafe cross-thread ncurses calls. The solution uses three mechanisms working together:

- `tui_set_notification()` (background thread): writes the message into a mutex-protected global buffer. It makes no ncurses calls whatsoever.
- `tui_getch()` (main thread): uses `wtimeout(win, 300)` instead of blocking indefinitely. Every 300 ms it wakes up, checks the mutex buffer, and if a message is present, calls `tui_modal_notification()` before resuming.
- `_modal_info()` dismissal: uses a plain `wgetch(pop)` bookended by two `flushinp()` calls — one before (to discard stale buffered keys) and one after (to discard the dismissal key) — preventing any keypress from propagating to the underlying window.

Security/correctness note

The previous implementation called `ungetch(KEY_F(12))` from the background thread. ncurses is not thread-safe; `ungetch` from a non-main thread corrupts the input buffer and causes dropped or phantom keypresses. The polling approach is the only correct solution.

Modal Dialog Family

All modals (success, error, confirm, notification, menu, pick-item) share the same `_modal_info()` core or a common pattern. They are all self-sizing based on content width, word-wrapped using `tui_print_wrapped()`, and restored via `clearok(curscr, TRUE) + refresh()` to repaint any windows that were behind them.

3.6 utils/hashing_utils.c — Password Security

Implements the complete password lifecycle: random salt generation, salted hashing, and verification. The stored format is `salt_hex$hash_hex` — the 16-byte salt and 32-byte SHA-256 digest stored as hexadecimal strings separated by a dollar sign.

Function	Implementation detail
<code>generateSalt()</code>	Calls <code>RAND_bytes()</code> (OpenSSL CSPRNG). Falls back to <code>rand()</code> seeded with <code>time()</code> only if OpenSSL fails — this fallback is weak and should be flagged for removal in production.
<code>hashPassword()</code>	Uses OpenSSL <code>EVP_DigestInit/Update/Final</code> — the modern EVP API rather than the deprecated <code>SHA256()</code> direct call, ensuring forward compatibility.
<code>verifyPassword()</code>	Extracts the salt from the stored string, re-hashes the candidate password with the same salt, and compares with <code>strcmp()</code> . Comparison is not constant-time — a timing-safe <code>memcmp</code> should be used in production.

3.7 `utils/ipc_utils.c` — Inter-Process / Inter-Thread Communication

Implements a named FIFO (`mkfifo`) per user located at `/tmp/atm_fifo_<username_lowercase>`. This allows any process or thread on the same machine to deliver a notification to a logged-in user.

- `createUserPipe()`: creates the FIFO if it does not exist; tolerates `EEXIST` gracefully.
- `notifyUser()`: opens the FIFO with `O_WRONLY | O_NONBLOCK` — non-blocking so the sender is never stalled if the recipient is not reading.
- `notificationListener()`: runs on a background pthread; opens the FIFO `O_RDONLY | O_NONBLOCK`; polls with `read()` in a 1-second sleep loop; on receipt calls `tui_set_notification()` which only touches the mutex-protected buffer, never ncurses.

Robustness The non-blocking writer is critical. Without `O_NONBLOCK`, `transferOwnership()` would block indefinitely if the recipient is not currently logged in, freezing the sender's UI.

3.8 `validation/validators.c` — Pure Input Validators

All validators follow an identical contract: they take the raw input string, write a human-readable error into a caller-supplied `char[128]` buffer, and return 1 (valid) or 0 (invalid). They perform no I/O, make no ncurses calls, and have no side effects. This design means they can be unit-tested in isolation and reused from both the legacy printf-based flow and the ncurses TUI.

Validator	Rules enforced
<code>val_account_number()</code>	Digits only, max 10, positive, not already in the user's existing ID list.

val_date()	MM/DD/YYYY format, month 1-12, year 2000-2100, correct days per month including leap years.
val_country()	Letters/digits/spaces/hyphens only; must match an entry in data/countries.txt by name, 2-letter ISO, or 3-letter ISO code.
val_phone()	8-10 digits, no separators.
val_amount()	Positive decimal, comma rejected, at most 2 decimal places, no non-numeric characters.
val_account_type()	Must be one of: savings, current, fixed01, fixed02, fixed03.

3.9 database/database.c — Schema Initialisation

initialize_database() runs on every startup. It uses CREATE TABLE IF NOT EXISTS, meaning it is fully idempotent — running it on an already-initialised database is a no-op. The users table stores name, salted-hash password, and an auto-increment ID. The accounts table stores all account fields with a foreign key reference to users.id.

```

users:
  id          INTEGER PRIMARY KEY AUTOINCREMENT
  name        TEXT NOT NULL
  password    TEXT NOT NULL           -- format: salt_hex$hash_hex

accounts:
  id          INTEGER PRIMARY KEY AUTOINCREMENT
  user_id     INTEGER NOT NULL        -- FK -> users.id
  account_id  INTEGER NOT NULL        -- user-chosen number
  creation_date TEXT NOT NULL        -- ISO 8601: YYYY-MM-DD
  country     TEXT
  phone       TEXT
  balance     REAL DEFAULT 0.0
  account_type TEXT                  -- savings|current|fixed01|02|03

```

4. Security Analysis

4.1 Strengths

- Salted SHA-256 passwords: each user has a unique 16-byte cryptographically random salt, preventing rainbow-table attacks and ensuring that two users with the same password have different stored hashes.
- Parameterised SQL queries throughout: all database queries use sqlite3_bind_text / sqlite3_bind_int. There is no string concatenation into SQL — SQL injection is architecturally impossible.
- Case-insensitive username lookups use LOWER() inside the SQL expression, not in application code, ensuring collation is handled consistently by SQLite.
- Password input masks characters with asterisks inside the ncurses input field; the raw password string is never written to any log or file.

- The IPC writer uses `O_NONBLOCK` to prevent a malicious or absent recipient from stalling the sender.

4.2 Observations for Future Hardening

- `verifyPassword()` uses `strcmp()` for hash comparison. In security-critical contexts a constant-time comparison (e.g. `CRYPTO_memcmp` from OpenSSL) should be used to prevent timing side-channel attacks.
- The `generateSalt()` fallback to `rand()+time()` is a weak PRNG. The fallback branch should be replaced with an abort or error return to ensure salt generation always uses the CSPRNG.
- The database file `atm.db` has no encryption at rest. Sensitive account data could be read by any OS user with file-system access.
- There is no account lockout after repeated failed login attempts, leaving the application open to brute-force password guessing.

5. Distinct Robustness Features

The following design decisions elevate this project beyond a typical student implementation:

5.1 Thread-Safe Notification Architecture

Most TUI applications that add background notification support do so by calling `ncurses` functions from a background thread, which is undefined behaviour. This project explicitly avoids that by confining all `ncurses` calls to the main thread. The background thread communicates exclusively through a mutex-protected string buffer, and the main thread polls it via `wtimeout()`. This is the only correct approach and is the same pattern used by production terminal applications.

5.2 Input Bleed Prevention

Modal dialogs use a pre-flush before waiting for dismissal and a post-flush after. This two-sided flush ensures that a keypress that arrives while a modal is being drawn does not accidentally dismiss it immediately, and that the dismissal key does not propagate to the underlying menu and trigger an unwanted selection.

5.3 Pure Validators with Structured Error Strings

Every validation function writes a human-readable error string into a caller-supplied buffer rather than printing to `stdout`. This makes validators testable in isolation, reusable across UI contexts (both the legacy terminal flow and the `ncurses` flow use the same validators), and keeps error messaging co-located with rendering logic rather than scattered through business logic.

5.4 Idempotent Database Initialisation

The database schema is created with IF NOT EXISTS on every startup. This means the application can be run on a fresh machine, on an already-initialised machine, or after a partial failure without any schema corruption or need for a separate setup step.

5.5 Zero-Balance Account Deletion Enforcement

`removeAccount()` queries the current balance from the database at the time of deletion, not from a cached value. It refuses deletion if `balance > 0.0` and shows the exact balance in the error message. A separate confirmation modal (`tui_modal_confirm`) is displayed before the DELETE statement executes.

5.6 UTF-8 Aware Layout Engine

`tui_dispw()` decodes UTF-8 byte sequences to determine visual display width. Without this, any username, country name, or message containing multi-byte characters (accented letters, CJK, emoji) would cause misaligned borders, truncated text, or off-centre titles. This is a rare correctness feature for a C terminal application.

5.7 Non-Blocking IPC Sender

`notifyUser()` opens the recipient FIFO with `O_WRONLY | O_NONBLOCK`. If the recipient is offline (FIFO has no reader), the `open()` call fails immediately and is logged to `/tmp/atm_debug.log`. The sender's operation completes normally. Without this flag, `transferOwnership()` would hang indefinitely when transferring to an offline user.

6. Interest Rate Model

Account Type	Rate	Term	Interest Calculation
savings	7% / yr	Rolling	balance $\times$ 0.07 / 12 per month, shown on deposit day.
current	0%	N/A	No interest generated.
fixed01	4% / yr	1 year	balance $\times$ 0.04 $\times$ 1, payable on maturity date.
fixed02	5% / yr	2 years	balance $\times$ 0.05 $\times$ 2, payable on maturity date.
fixed03	8% / yr	3 years	balance $\times$ 0.08 $\times$ 3, payable on maturity date.

Fixed-term accounts block all transactions (deposit and withdrawal) for the lifetime of the account. This is enforced in `makeTransaction()` by checking the account type before displaying the amount form.

7. Known Limitations and Recommended Improvements

Limitation	Recommended Fix
<code>auth.c</code> is dead code alongside <code>init_menu.c</code>	Remove <code>auth.c</code> and promote <code>getPassword()</code> / <code>isUsernameTaken()</code> into a shared <code>auth_db.c</code> module.
No session timeout	Add a configurable idle timer that logs out the user after N minutes of inactivity.
No account lockout after failed logins	Track failure count per username in the DB; lock after 5 attempts.
<code>strcmp</code> -based hash comparison	Replace with <code>CRYPTO_memcmp()</code> or equivalent constant-time comparison.
Notification listener polls at 1-second intervals	Replace <code>sleep(1)</code> with <code>select()</code> or a self-pipe trick to wake immediately on message arrival without busy-waiting.
No transaction history	Add a transactions table recording every deposit, withdrawal, and transfer with timestamp.
Country lookup is $O(n)$ file scan	Load <code>countries.txt</code> into a hash map at startup for $O(1)$ average lookup.

8. Conclusion

The Bank Management System demonstrates a high standard of software engineering for a C terminal application. Its layered architecture separates concerns cleanly, its validator framework is reusable and testable, and its notification subsystem is genuinely thread-correct — a distinction that places it ahead of most comparable projects. The security foundation (salted hashing, parameterised queries, masked input) is solid, and the identified gaps are well-understood and tractable.

The codebase is suitable as a teaching reference for multi-file C project structure, database-backed application design, ncurses TUI development, and POSIX threading, as well as a foundation for a production-grade banking terminal application with targeted hardening.

Summary Clean layered architecture, correct threaded notification system, parameterised SQL throughout, salted SHA-256 passwords, and a pure reusable validation framework.

Recommended improvements: remove dead auth.c, add constant-time hash comparison, add account lockout, and introduce a transaction history table.